
Table of Contents

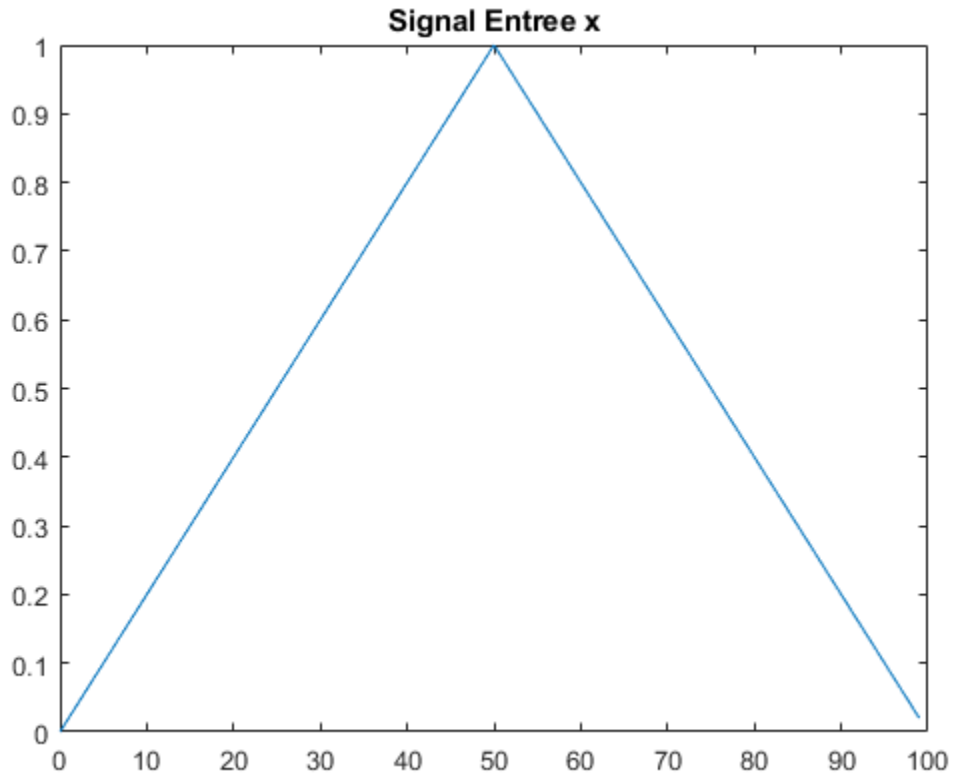
signal	1
filtres	2
convolution naïve (sans bruit)	3
signal quantifier	4
methode matricielle	6
Déconvolution naïve (sans bruit)	8
Déconvolution avec bruit	8
Déconvolution par Moindres Carrés	9
Tracé du signal déconvolué	10
Calcul de l'énergie de l'erreur de reconstruction	11
Affichage de l'erreur	12
Décomposition en valeurs singulières (SVD) de H	12
Tracé des valeurs singulières	12
Calcul du conditionnement de H	13
quadratique	13

signal

```
Lx = 100;

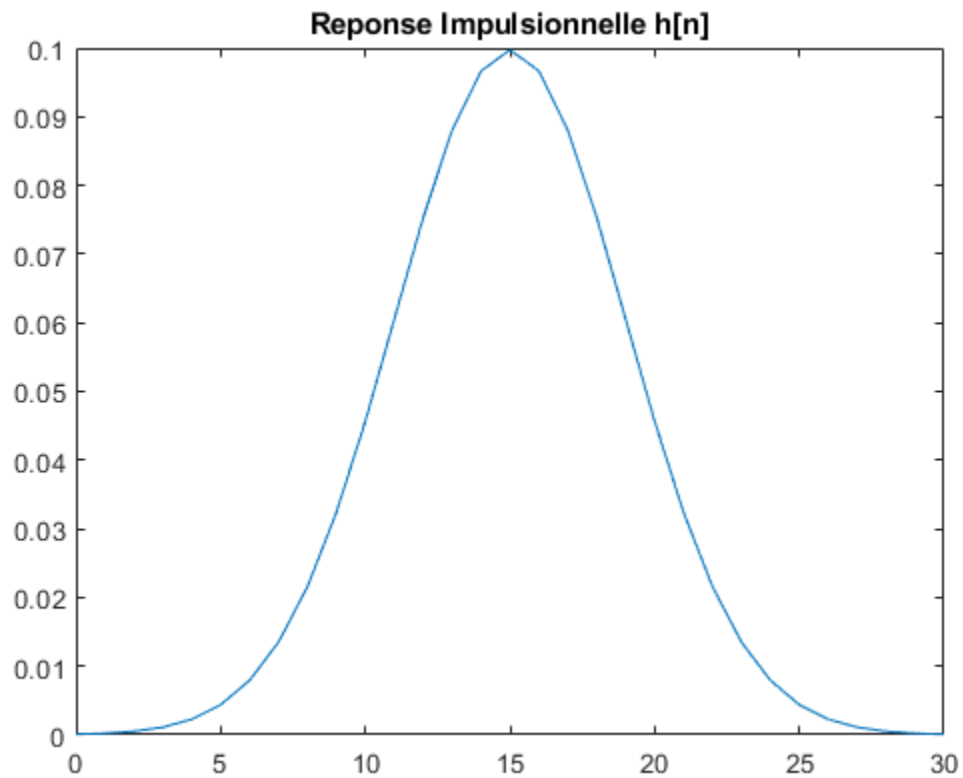
t1 = 0:Lx/2 - 1;
x1 = t1 / (Lx/2);

t2 = Lx/2:Lx - 1;
x2 = (-t2 + Lx) / (Lx/2);
tx=[t1 t2];
x = [x1 x2];
figure(1);
plot(tx, x);
title('Signal Entree x');
```



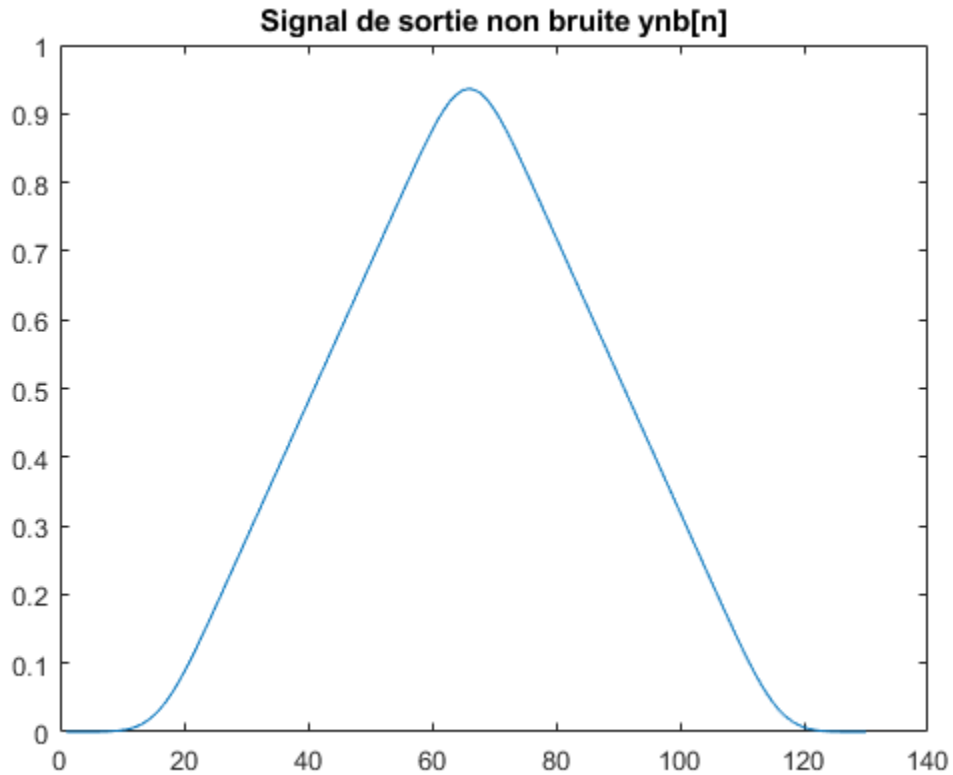
filtres

```
m= 15;  
sigmah = 4;  
t = 0:30;  
  
% determiner la reponse impulsionnelle h[n] obtenue par l'equation (1.1)  
h_n = (1/(sigmah*sqrt(2*pi))) * exp((- (t-m).^2) / (2*sigmah.^2));  
  
figure(2);  
plot(t, h_n);  
title('Reponse Impulsionnelle h[n]');
```



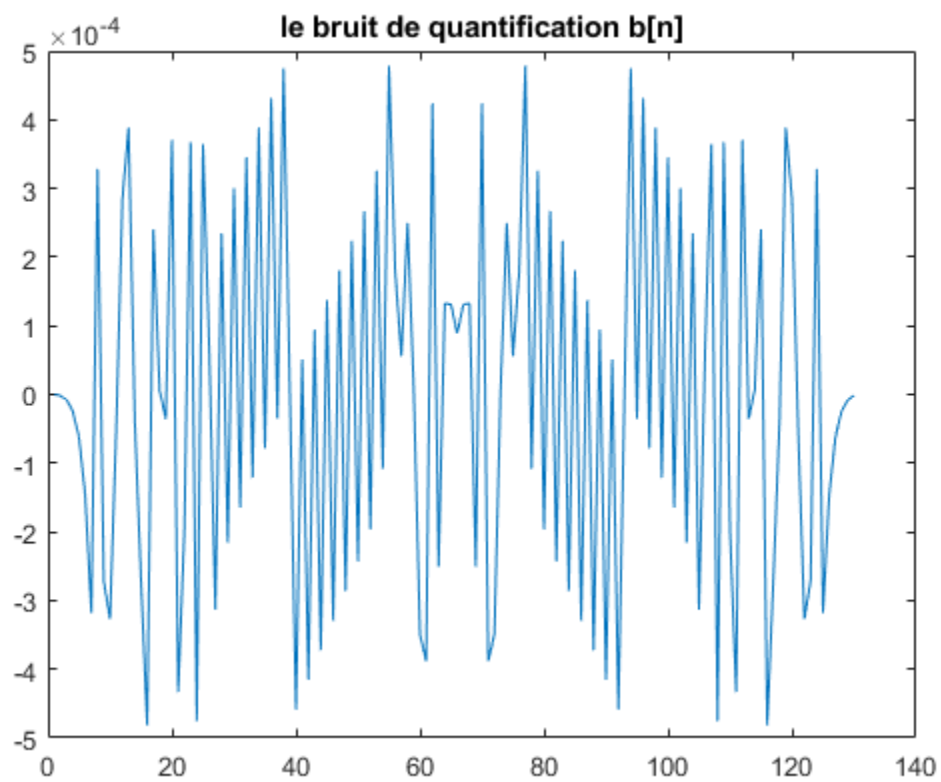
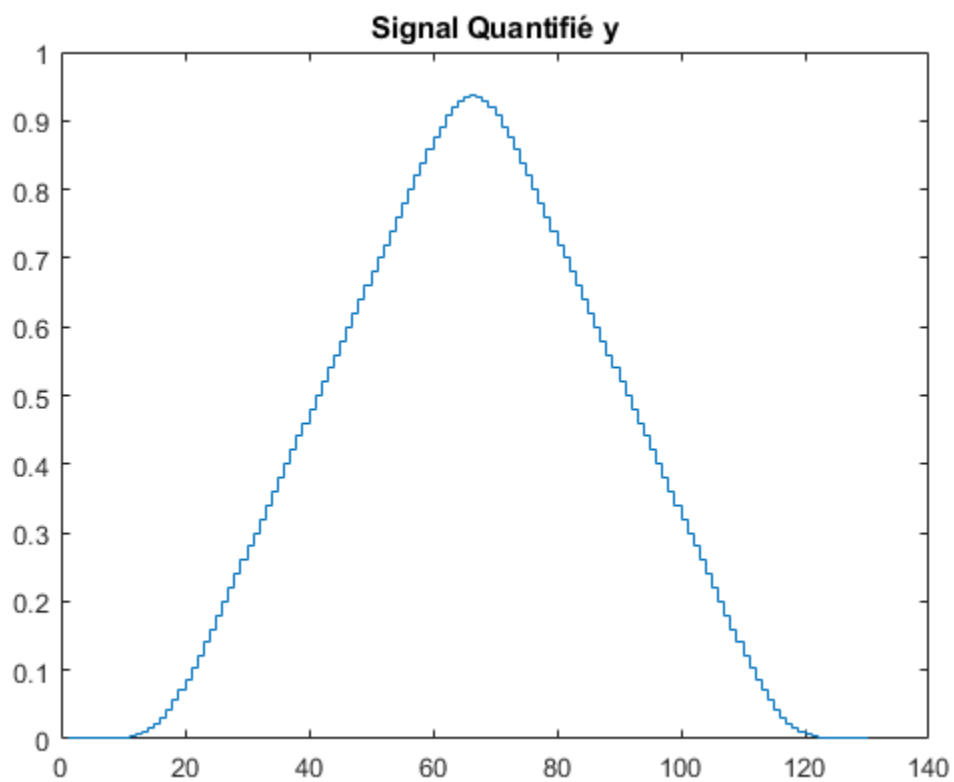
convolution naïve (sans bruit)

```
ynb_n = conv(x, h_n);  
figure(3);  
plot(ynb_n);  
title('Signal de sortie non bruité ynb[n]');
```



signal quantifier

```
y = round(ynb_n * 2^10) / 2^10;  
figure(4);  
stairs(y);  
title('Signal Quantifié y');  
b_n = y - ynb_n;  
figure(5);  
plot(b_n);  
title('le bruit de quantification b[n]');
```



methode matricielle

```
Ny=size(ynb_n,2);
Nx=size(x,2);
Nh=size(h_n,2);

H_ligne = zeros(1,Nx);
H_colones = zeros(Ny,1);
H= toeplitz(H_colones,H_ligne);

    for i=1:Nx
        for j=1:31
            var = i + j - 1;
            H(var,i)=h_n(j);
        end
    end

% Pour adapter la dimension de H faut transposer la matrice x
x_t = transpose(x);

ynb_om = H * x_t;

t_om = 0:129;

figure(6);

plot(t_om, ynb_om);

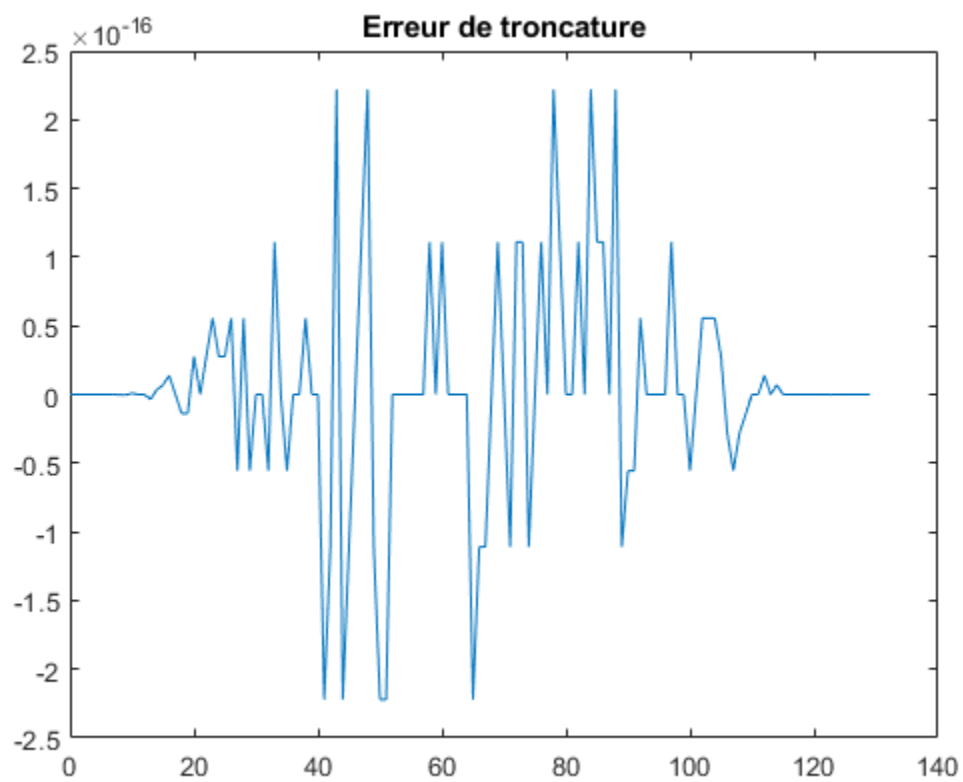
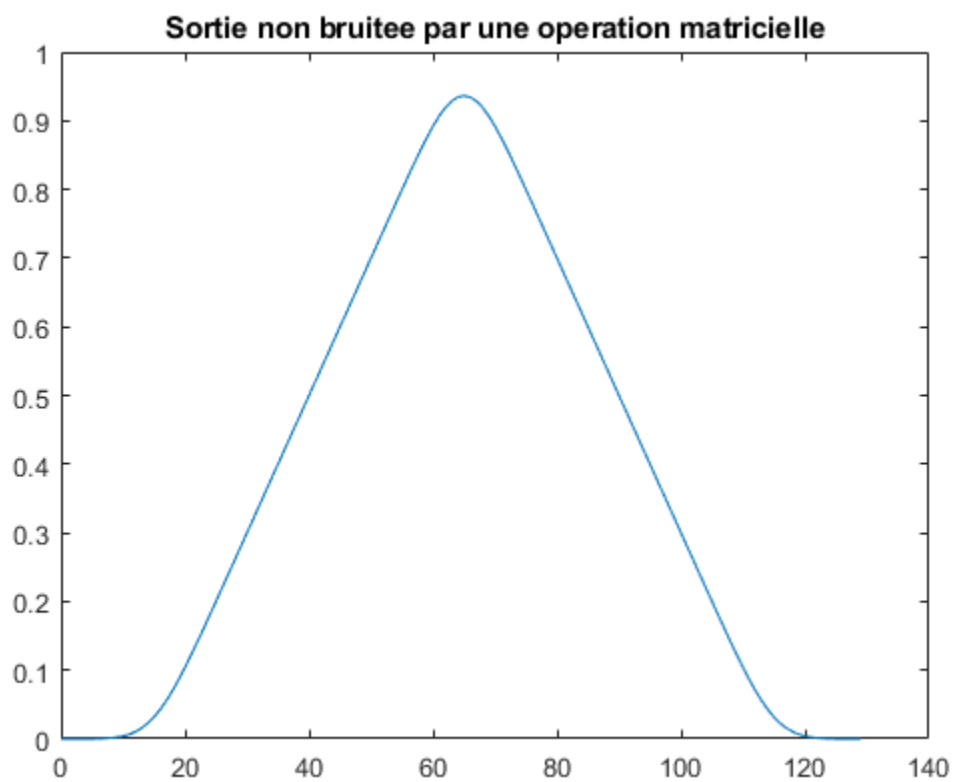
title('Sortie non bruitée par une operation matricielle');

% Pour adapter la dimension de ynb_n faut transposer la matrice ynb_om
Erreur_t = ynb_om' - ynb_n;

figure(7);

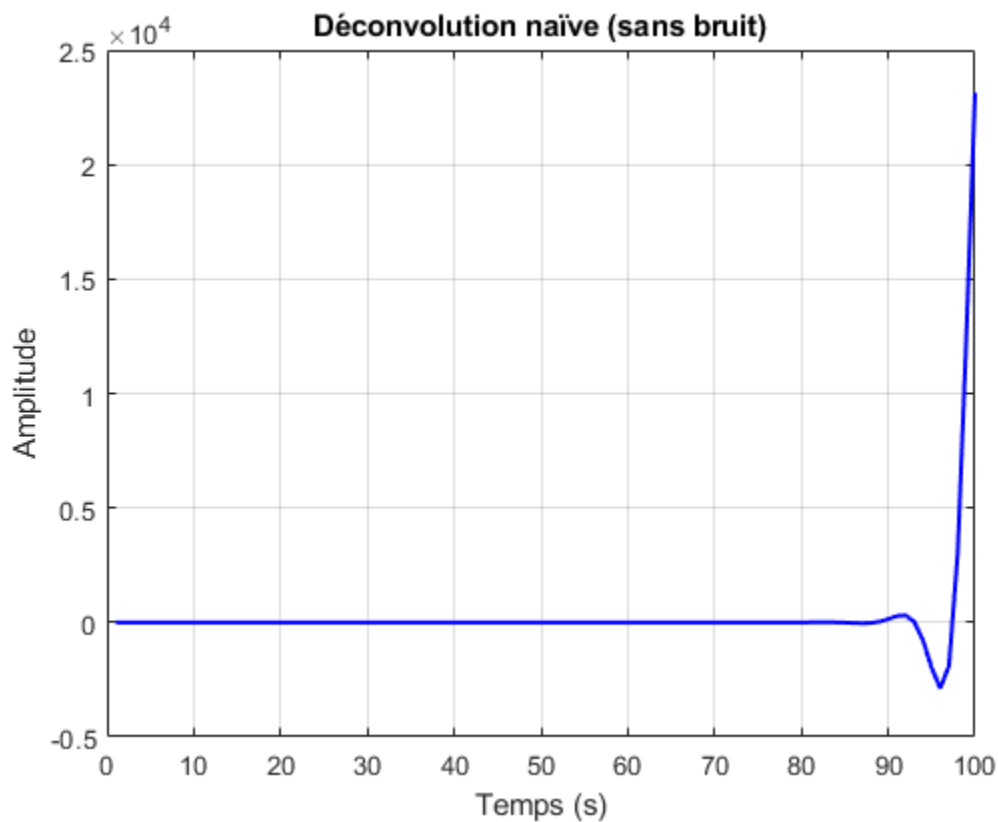
plot(t_om, Erreur_t);

title('Erreur de troncature');
```



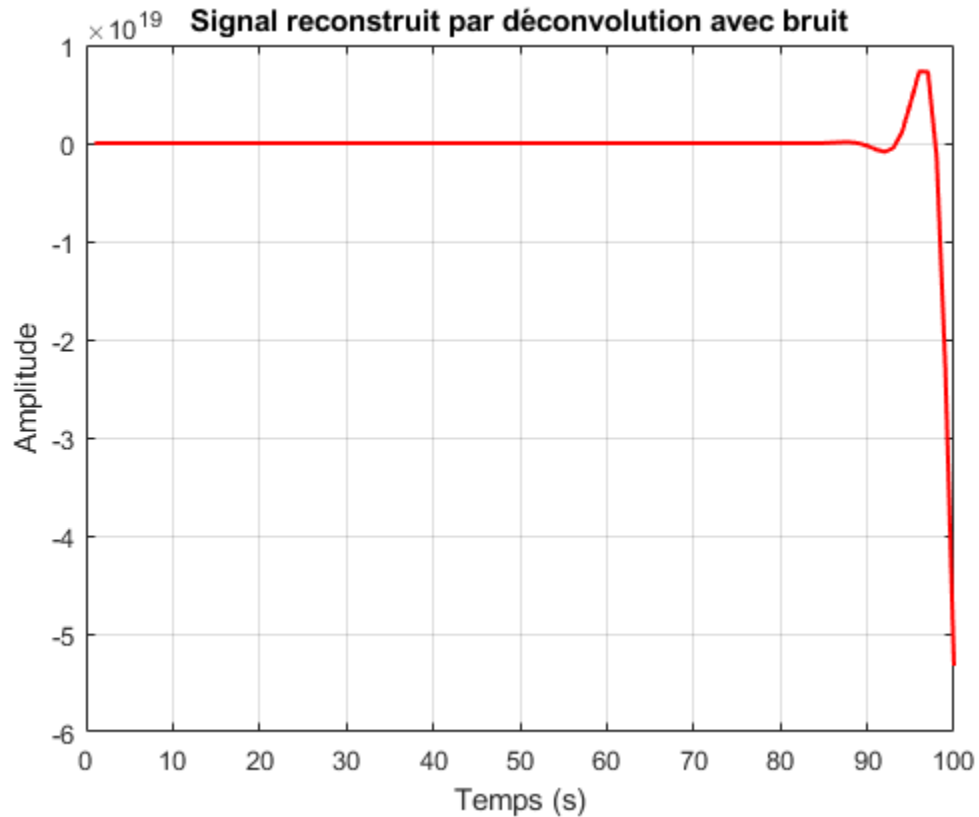
Déconvolution naïve (sans bruit)

```
x_rec = deconv(ynb_n, h_n);  
  
figure;  
plot(x_rec, 'b', 'LineWidth', 1.5);  
grid on;  
title('Déconvolution naïve (sans bruit)');  
xlabel('Temps (s)');  
ylabel('Amplitude');
```



Déconvolution avec bruit

```
[xrec, r] = deconv(y, h_n);  
  
figure;  
plot(xrec, 'r', 'LineWidth', 1.5);  
grid on;  
title('Signal reconstruit par déconvolution avec bruit');  
xlabel('Temps (s)');  
ylabel('Amplitude');
```

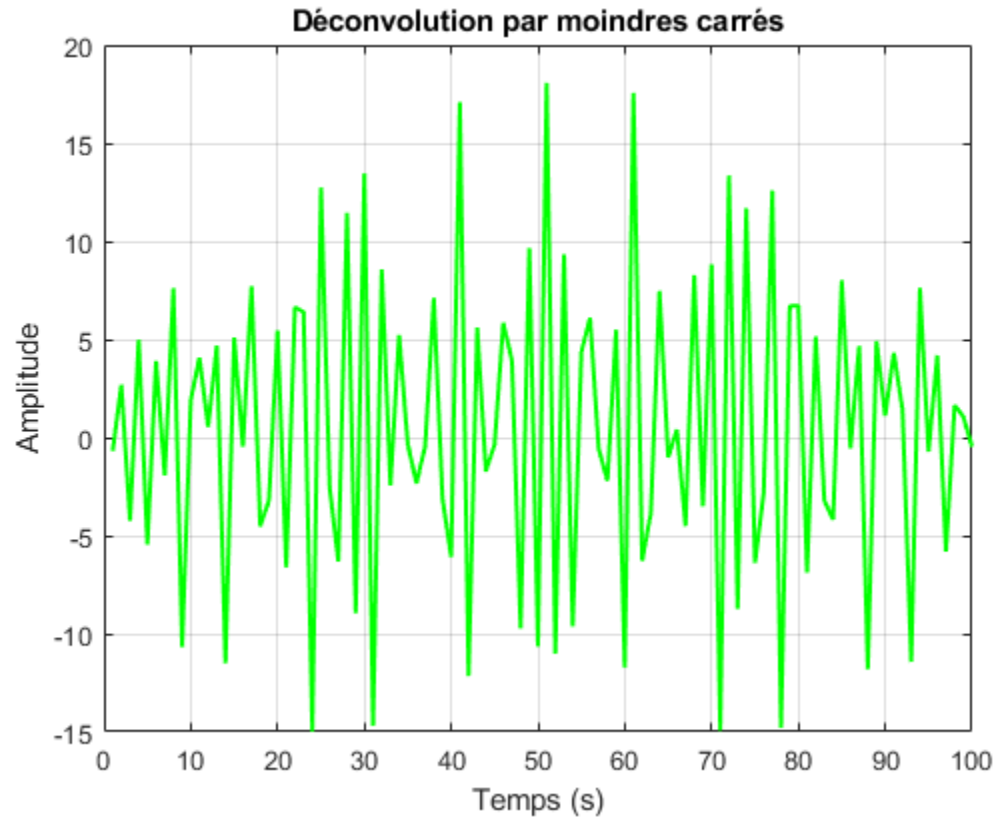



Déconvolution par Moindres Carrés

```
y = y(:); % Convertir y en vecteur colonne pour compatibilité avec  
l'algèbre matricielle
```

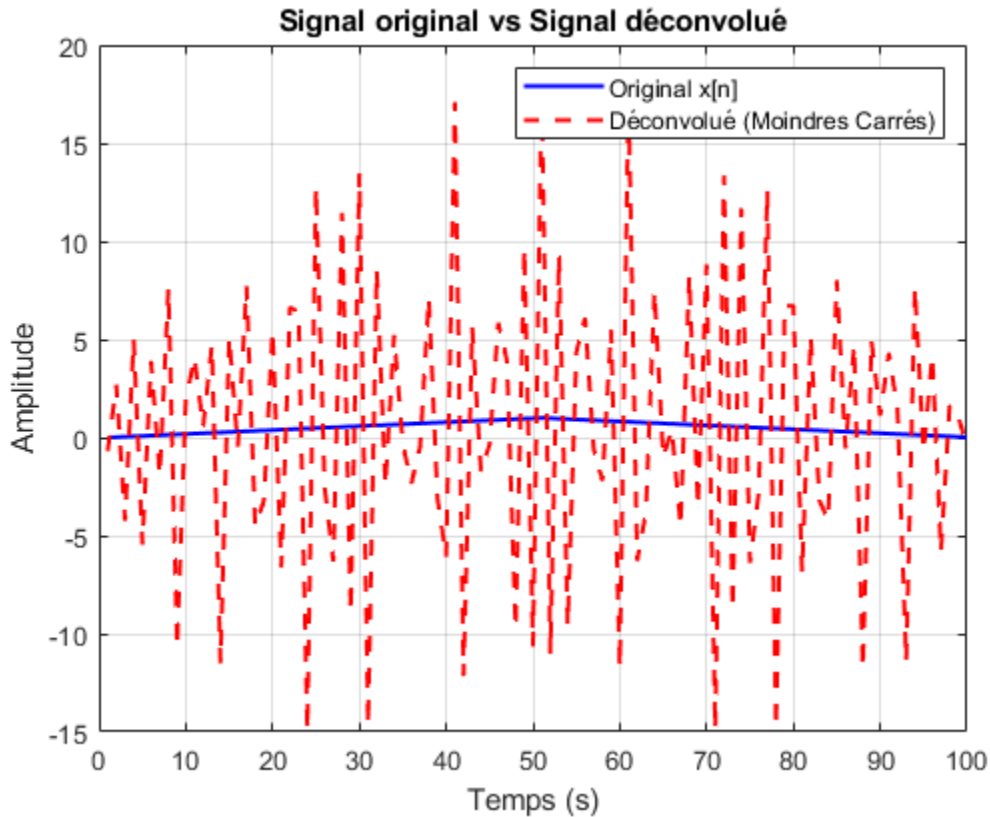
```
xrec_ls = H \ y; % Résolution du système  $H * x = y$ 
```

```
% Tracé du signal reconstruit  
figure;  
plot(xrec_ls, 'g', 'LineWidth', 1.5);  
grid on;  
title('Déconvolution par moindres carrés');  
xlabel('Temps (s)');  
ylabel('Amplitude');
```



Tracé du signal déconvolué

```
figure;  
plot(x, 'b', 'LineWidth', 1.5);  
hold on;  
plot(xrec_ls, 'r--', 'LineWidth', 1.5);  
hold off;  
legend('Original x[n]', 'Déconvolué (Moindres Carrés)');  
title('Signal original vs Signal déconvolué');  
xlabel('Temps (s)');  
ylabel('Amplitude');  
grid on;
```



Calcul de l'énergie de l'erreur de reconstruction

```
erreur_reconstruction = x - xrec_ls(1:length(x)); % Ajustement de la taille
si nécessaire
energie_erreur = sum(erreur_reconstruction.^2);
```

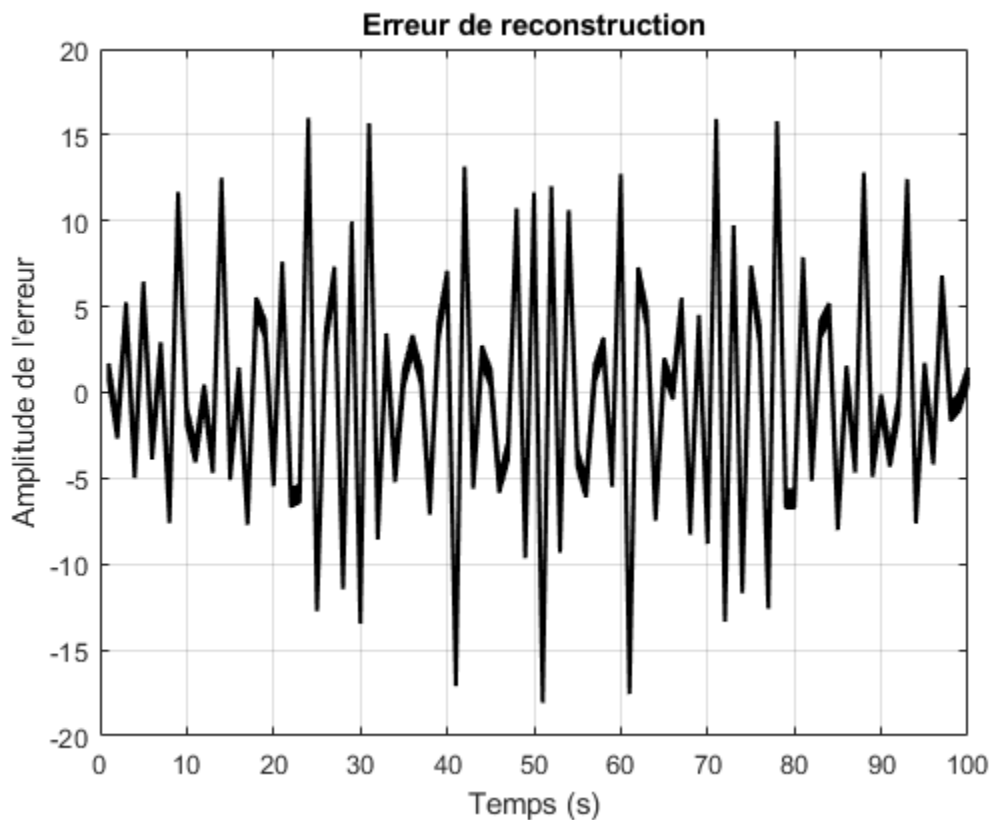
```
disp(['Énergie de l'erreur de reconstruction : ', num2str(energie_erreur)]);
```

```
Énergie de l'erreur de reconstruction : 6000.4581      5998.4981
5996.618      5994.818      5993.098      5991.458      5989.898
5988.418      5987.018      5985.698      5984.458      5983.298
5982.218      5981.218      5980.2979      5979.4579      5978.6979
5978.0179      5977.4179      5976.8979      5976.4579      5976.0979
5975.8179      5975.6179      5975.4979      5975.4579      5975.4978
5975.6178      5975.8178      5976.0978      5976.4578      5976.8978
5977.4178      5978.0178      5978.6978      5979.4578      5980.2978
5981.2178      5982.2177      5983.2977      5984.4577      5985.6977
5987.0177      5988.4177      5989.8977      5991.4577      5993.0977
5994.8177      5996.6177      5998.4976      6000.4576      5998.4976
5996.6177      5994.8177      5993.0977      5991.4577      5989.8977
5988.4177      5987.0177      5985.6977      5984.4577      5983.2977
5982.2177      5981.2178      5980.2978      5979.4578      5978.6978
5978.0178      5977.4178      5976.8978      5976.4578      5976.0978
5975.8178      5975.6178      5975.4978      5975.4579      5975.4979
```

5975.6179	5975.8179	5976.0979	5976.4579	5976.8979
5977.4179	5978.0179	5978.6979	5979.4579	5980.2979
5981.218	5982.218	5983.298	5984.458	5985.698
5987.018	5988.418	5989.898	5991.458	5993.098
5994.818	5996.618	5998.4981		

Affichage de l'erreur

```
figure;
plot(erreur_reconstruction, 'k', 'LineWidth', 1.5);
title('Erreur de reconstruction');
xlabel('Temps (s)');
ylabel('Amplitude de l'erreur');
grid on;
```



Décomposition en valeurs singulières (SVD) de H

```
singular_values = svd(H); % Calcul des valeurs singulières de H
```

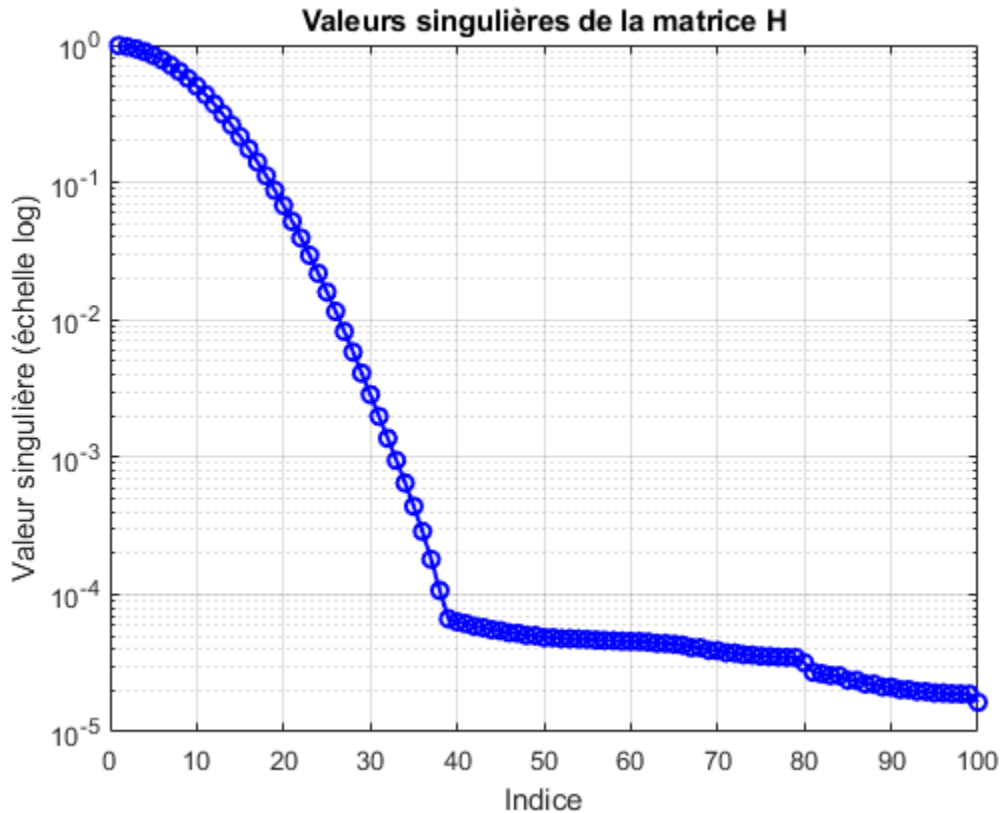
Tracé des valeurs singulières

```
figure;
semilogy(singular_values, 'bo-', 'LineWidth', 1.5);
```

```

title('Valeurs singulières de la matrice H');
xlabel('Indice');
ylabel('Valeur singulière (échelle log)');
grid on;

```



Calcul du conditionnement de H

```

sigma_max = max(singular_values); % Plus grande valeur singulière
sigma_min_non_nulle = min(singular_values(singular_values > 1e-10)); % Plus
petite valeur singulière non nulle

```

```

conditionnement_H = sigma_max / sigma_min_non_nulle;
disp(['Conditionnement de H : ', num2str(conditionnement_H)]);

```

Conditionnement de H : 60407.4906

quadratique

```

lambda_values = logspace(-7, 2, 10); % Génération de 10 valeurs
logarithmiquement espacées
errors = zeros(size(lambda_values)); % Stocker l'erreur de reconstruction

```

```

% Définition de la matrice D1
Nx = size(H, 2);
dcoll = zeros(Nx, 1);

```

```

dcoll(1:2) = [1; -1];
dlig1 = zeros(1, Nx);
dlig1(1,1) = dcoll(1,1);
D1 = toeplitz(dcoll, dlig1);

% Boucle sur les différentes valeurs de lambda
for i = 1:length(lambda_values)
    lambda = lambda_values(i);
    x_reg = (H' * H + lambda * (D1' * D1)) \ (H' * y); % Résolution

    % Calcul de l'erreur de reconstruction
    errors(i) = norm(x - x_reg, 2); % Erreur L2 entre x réel et x reconstruit
end

% Trouver le lambda optimal
[~, best_idx] = min(errors);
best_lambda = lambda_values(best_idx);

% Affichage du meilleur lambda
disp(['Meilleur lambda : ', num2str(best_lambda)]);

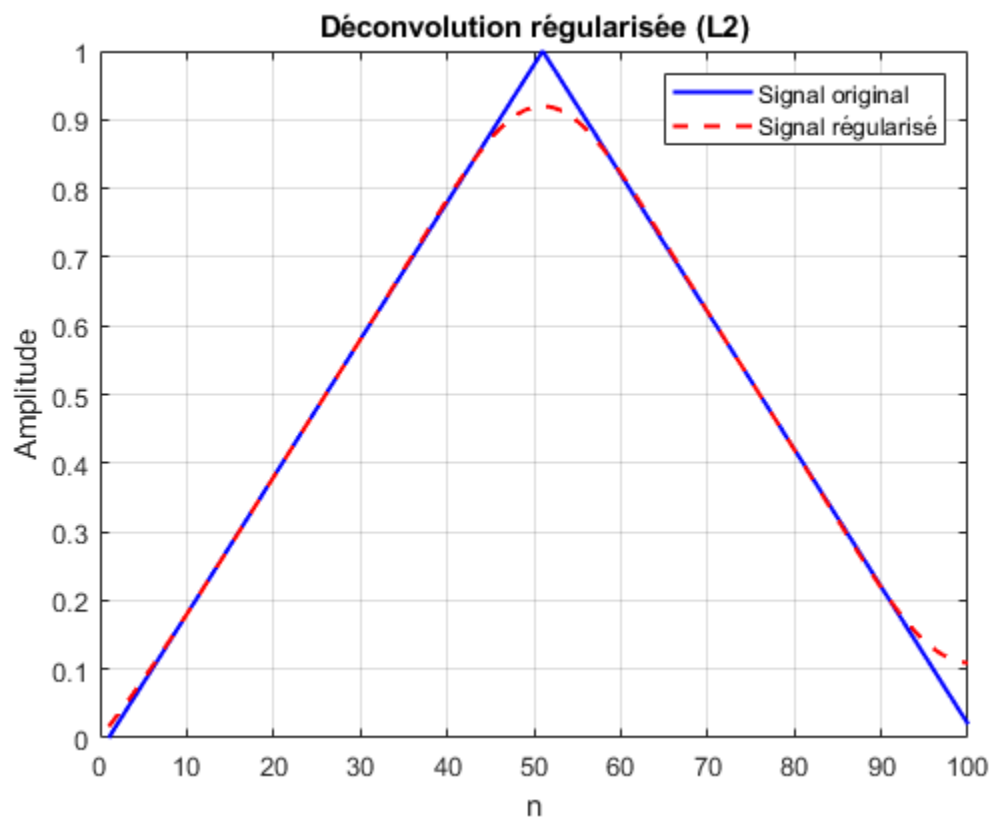
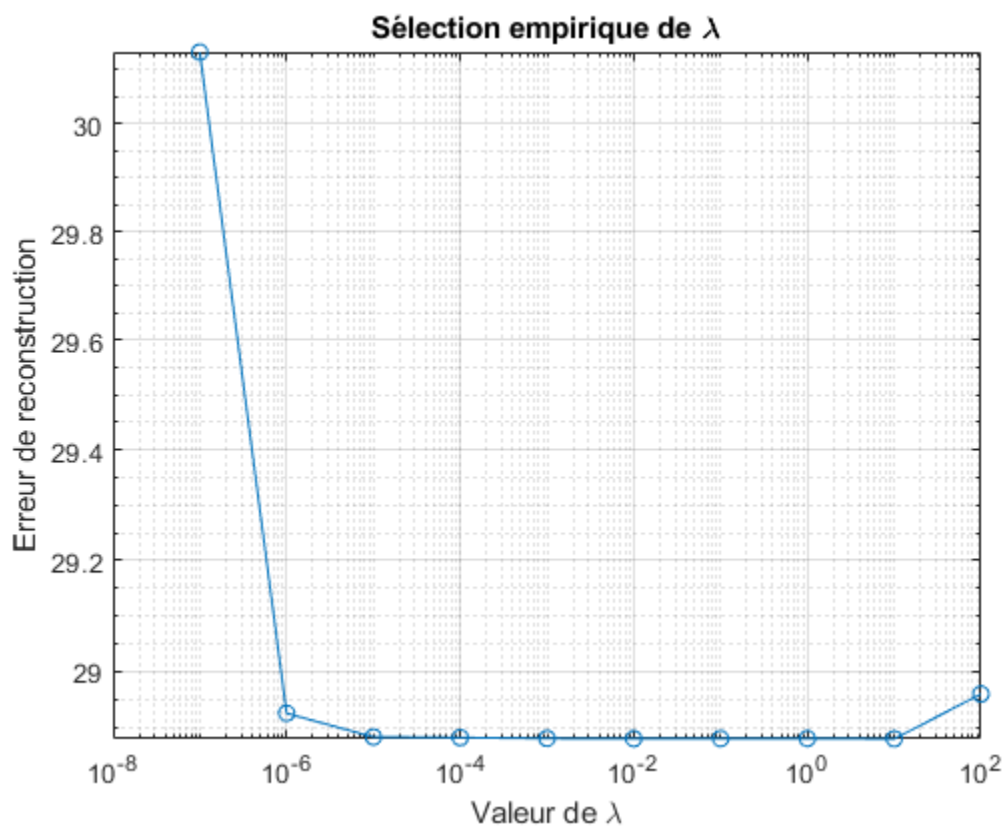
% Tracer l'évolution de l'erreur
figure;
loglog(lambda_values, errors, '-o');
xlabel('Valeur de \lambda');
ylabel('Erreur de reconstruction');
title('Sélection empirique de \lambda');
grid on;

% Reconstruire le signal avec le meilleur lambda
x_optimal = (H' * H + best_lambda * (D1' * D1)) \ (H' * y);

% Tracer le signal reconstruit
figure;
plot(x, 'b', 'LineWidth', 1.5); hold on;
plot(x_optimal, 'r--', 'LineWidth', 1.5);
legend('Signal original', 'Signal régularisé');
xlabel('n');
ylabel('Amplitude');
title('Déconvolution régularisée (L2)');
grid on;

Meilleur lambda : 10

```



Published with MATLAB® R2024a